

1. Description de la méthode :

a. Prototype

```
private bool IsOnTheZone(GMapPolygon zone, PointLatLng point)
```

b. Résumé

Cette méthode vérifie si le point passé en paramètre est dans la zone passé en paramètre

c. Paramètres d'entrés

Paramètre d'entrée	Type	Description
zone	GMapPolygon	zone a testé
point	PointLatLng	point a testé

d. Valeur de retour

Cette méthode retourne true si point est à l'intérieur de zone

2. Vérification de l'appartenance d'un point à un polygone par produit scalaire

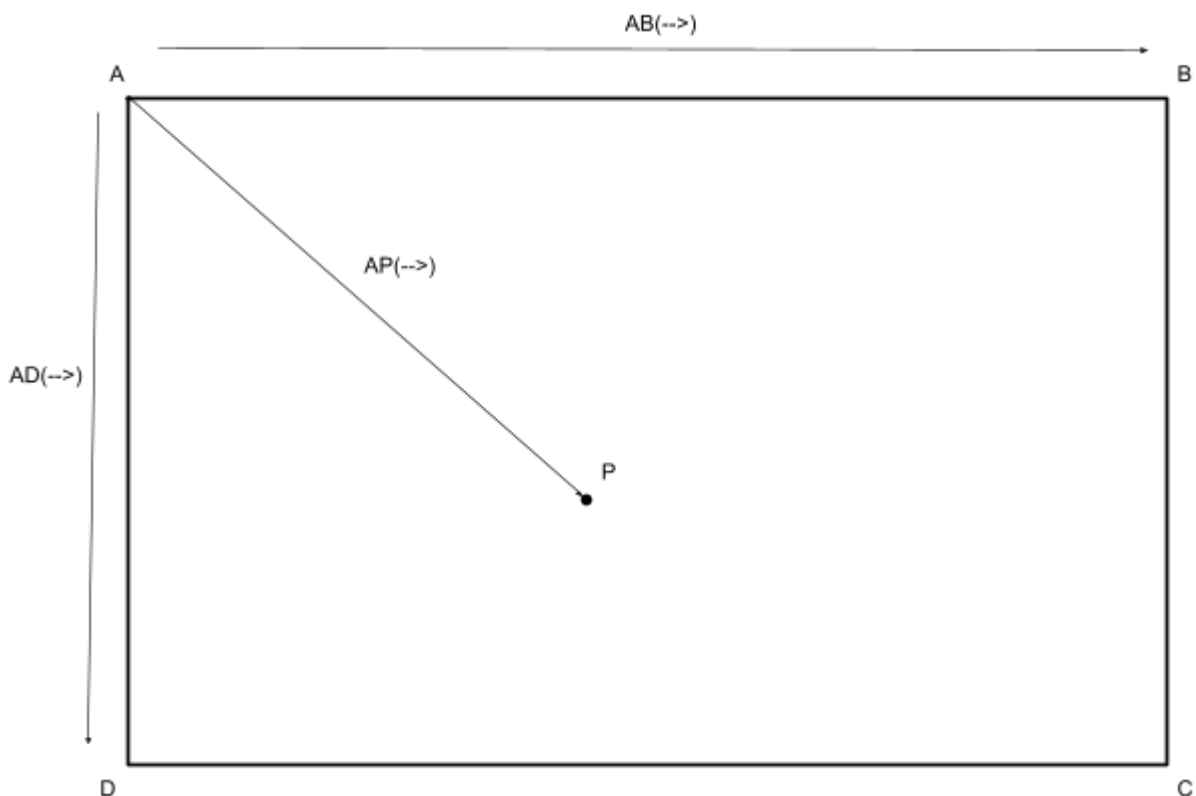
**Avant de vérifier si le point est à l'intérieur de la zone j'ai du projeté les sommets de la zone et le point sur un plan 2D à l'aide de la méthode LatLngToPoint (pour plus d'information allez voir la doc de cette méthode)*

Comme expliquer par le titre, **cette partie va expliquer comment la méthode IsOnTheZone vérifie si un point est à l'intérieur d'une zone/ d'un polygone**

Pour faire cela la méthode procède en deux étapes :

a. Création des vecteurs

Tout d'abord la méthode **créer des vecteurs à partir du point testé P et des deux coté du polygone AB et AD** comme ceci :



Les vecteur AB, AD et AP sont créés grâce à ces formules:

Documentation méthode IsOnTheZone

$$AP = (x_p - x_A, y_p - y_A)$$

$$AB = (x_B - x_A, y_B - y_A)$$

$$AD = (x_D - x_A, y_D - y_A)$$

Dans le code source de la méthode, cette partie est implémenté comme ceci :

```
//Creation des vecteurs
System.Windows.Point AB = new System.Windows.Point(list_XY[0].X - list_XY[1].X, list_XY[0].Y - list_XY[1].Y);
System.Windows.Point AD = new System.Windows.Point(list_XY[2].X - list_XY[1].X, list_XY[2].Y - list_XY[1].Y);
System.Windows.Point AP = new System.Windows.Point(point_mesure.X - list_XY[1].X, point_mesure.Y - list_XY[1].Y);
```

**list_XY est la liste des points projetée par la méthode LatLngToPoint représentant les sommets de la zone passé en paramètre. Les sommets ont été convertie dans cette ordre :*

Emplacement dans list_XY	Sommets
list_XY[0]	B
list_XY[1]	A
list_XY[2]	D
list_XY[3]	C

b. Vérification de la direction du vecteur AP

Après avoir créer les vecteurs **la méthode va comparer le vecteur AP avec les vecteurs AB et AD** afin de **vérifier si P est bien dans la zone**

Pour ce faire, elle va **projeter le vecteur AP sur le vecteur AB puis sur le vecteur AD** en effectuant ce que l'on appelle **un produit scalaire**

Si l'on considère la formule d'un produit scalaire comme étant celle ci :

$$AP * AB = ||AP|| * ||AB|| \cos(\theta)$$

Alors on peut définir **la formule de projection du vecteur AP sur AB** :

$$\frac{AP * AB}{||AB||} = ||AP|| * \cos(\theta)$$

Grâce à ce calcul cela nous donne la condition à respecter pour que P soit dans ABCD :

$$0 < \frac{AP * AB}{||AB||} < ||AB||$$

Ce qui peut être simplifiée en multipliant tout par $||AB||$:

Documentation méthode IsOnTheZone

$$0 < AP * AB < ||AB||^2$$

ou cette formule pour comparer avec AD

$$0 < AP * AD < ||AD||^2$$

Cette condition permettra de vérifier si **la projection du vecteur AP sur AB ou AD est bien compris dans sa norme** (soit est ce que le point est bien dans le rectangle) (**partie entouré en rouge**) **et si il est bien dans la bonne direction** (**partie entouré en bleu**)

Pour que le point soit valide il faut que les deux conditions soit respecté (le point C étant la valeur maximum valide)

Dans le code de source de notre méthode, cette condition est testé ici :

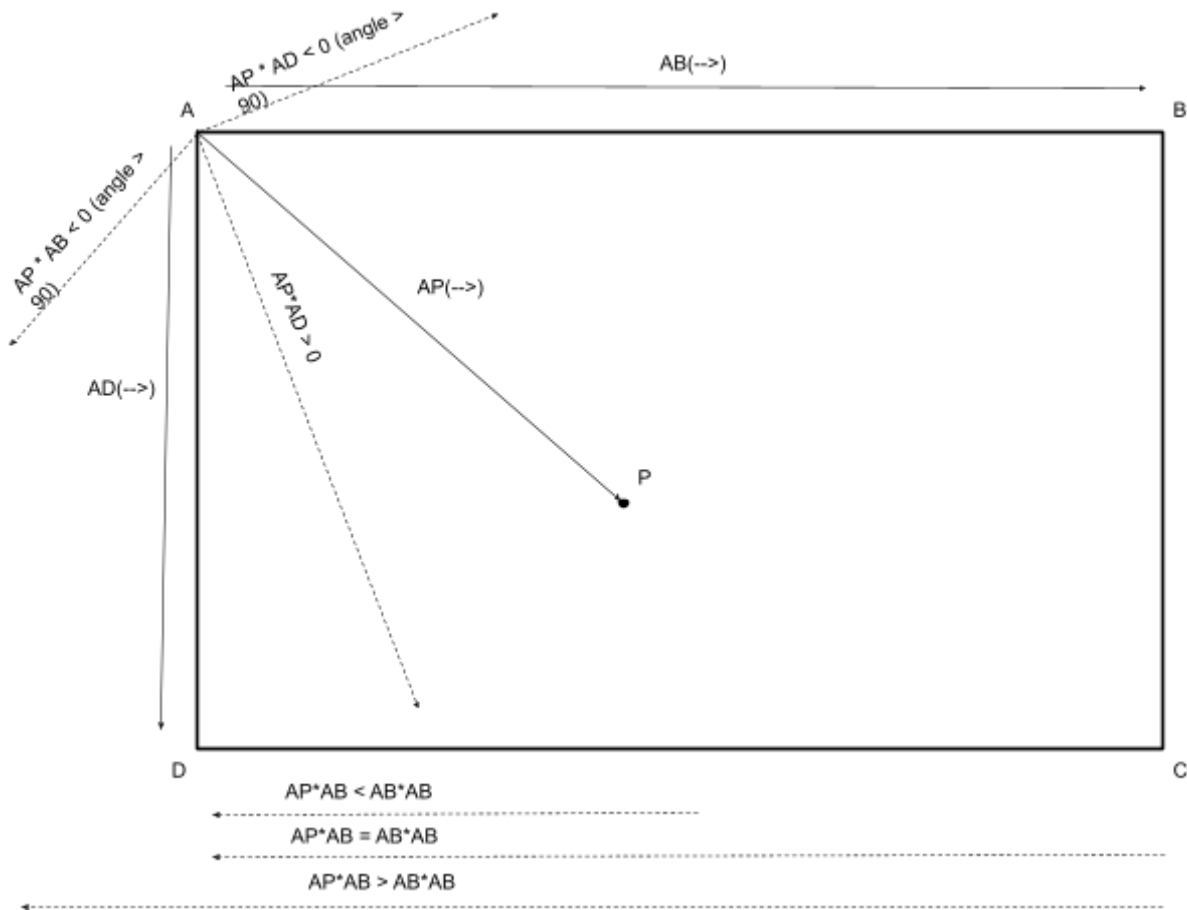
```
//Calcule des produit scalaire
double dotABAP = AP.X * AB.X + AP.Y * AB.Y;
double dotADAP = AP.X * AD.X + AP.Y * AD.Y;

double dotABAB = AB.X * AB.X + AB.Y * AB.Y;
double dotADAD = AD.X * AD.X + AD.Y * AD.Y;

//verification de la validité du point (Par produit scalaire)
if (dotABAP >= 0 && dotABAP <= dotABAB && dotADAP >= 0 && dotADAP <= dotADAD)
{
    is_valide = true;
}
```

c. Résumé

Cette partie est un petit résumé avec un schéma afin de mieux visualiser les résultats possible des produits scalaires



Valeur fausse	Description
$AP * AB < 0$	Le point P est a gauche du rectangle
$AP * AB > AB * AB$	Le point P est a droite du rectangle
$AP * AD < 0$	Le point P est au dessus du rectangle
$AP * AD > AD * AD$	Le point P est en dessous du rectangle